

ICT Class 8 - Formative Assessment 1

Formative Assessment (FA-1) - Answer Key

- Class 8

Class: Class 8

Assessment Type: Formative Assessment (FA-1)

Total Marks: 30

Duration: 90 minutes

Topics Covered: Functions

Curated and developed by

Hoysala T, Computer Science Teacher

MDRS SC-32 Bahaddurghatta (Kogunde), Chitradurga Taluk & District - 577519

CC BY-SA 4.0 - Free to reuse with attribution - 2026

Answer Key - Grading Guidelines

Note to Teacher: This answer key provides expected answers and marking guidelines. Accept equivalent answers where appropriate. Partial marks may be awarded for incomplete but correct responses.

Part A: Written Concept Paper (20 Marks)

Section I: Multiple Choice Questions (5 x 1 = 5 marks)

Q.No	Answer	Explanation
1	a	The function returns "Hello, " + name, so greet("Alice") returns "Hello, Alice"
2	c	The def keyword is used to define a function in Python
3	b	A pure function is one that returns a value and has no side effects
4	c	Python functions return None by default if no return statement is present
5	a	Parentheses () are used to call a function in Python

Section II: Short Answer Questions (5 x 2 = 10 marks)

1. **Define a function. Give its syntax in Python.**

- **Answer:** A function is a reusable block of code that performs a specific task. Syntax: `def function_name(parameters):` followed by an indented block of code.
- **Marking:** 1 mark for correct definition, 1 mark for correct syntax.

2. **Differentiate between arguments and parameters with one example each.**

- **Answer:** Parameters are variables listed in the function definition (e.g., `def greet(name):` where `name` is a parameter). Arguments are actual values passed when calling the function (e.g., `greet("Alice")` where "Alice" is an argument).
- **Marking:** 1 mark for explaining parameters with example, 1 mark for explaining arguments with example.

3. **Write the output of the factorial function.**

- **Answer:** 120
- **Marking:** 2 marks for correct answer ($5! = 5 \times 4 \times 3 \times 2 \times 1 = 120$).

4. **Explain the difference between a local variable and a global variable inside a function.**

- **Answer:** A local variable is defined inside a function and can only be accessed within that function. A global variable is defined outside any function and can be accessed throughout the program. To modify a global variable inside a function, the `global` keyword must be used.
- **Marking:** 1 mark for explaining local variable, 1 mark for explaining global variable.

5. **What is the purpose of the return statement in a function? Give an example.**

- **Answer:** The return statement sends a value back to the caller of the function. Example: `def add(a, b): return a + b` - when called as `add(3, 5)`, it returns 8.
- **Marking:** 1 mark for explaining purpose, 1 mark for correct example.

Section III: Long Answer Questions (5 x 1 = 5 marks)

1. Write a Python function `is_prime(n)` that checks whether a given number `n` is a prime number or not.

• **Answer:**

```
def is_prime(n):
    """Check if a number is prime."""
    if n <= 1:
        return False
    for i in range(2, int(n**0.5) + 1):
        if n % i == 0:
            return False
    return True
```

```
# Test with 5 values
for num in [2, 3, 4, 17, 20]:
    print(f"{num} is prime: {is_prime(num)}")
```

• **Marking:** 1 mark for function definition, 1 mark for docstring and edge cases, 1 mark for correct logic, 1 mark for test calls, 1 mark for output display.

Part B: Hands-on Practical Lab Task (5 marks)

BMI Calculator Program:

• **Answer:**

```
def calculate_bmi(weight_kg, height_m):
    """Calculate BMI with input validation."""
    if weight_kg <= 0 or height_m <= 0:
        return "Invalid input: weight and height must be positive"
    bmi = weight_kg / (height_m ** 2)
    if bmi < 18.5:
        category = "Underweight"
    elif bmi < 25:
        category = "Normal"
    elif bmi < 30:
        category = "Overweight"
    else:
        category = "Obese"
    return bmi, category
```

```
# Test with 4 persons
persons = [
    ("Alice", 55, 1.65),
    ("Bob", 85, 1.75),
    ("Charlie", 45, 1.60),
    ("Diana", 100, 1.70)
]

print(f"{'Name':<10} {'BMI':<8} {'Category':<12}")
print("-" * 30)
for name, weight, height in persons:
```

```
bmi, category = calculate_bmi(weight, height)
print(f"{name:<10} {bmi:<8.1f} {category:<12}")
```

- **Marking:** 1 mark for function definition, 1 mark for input validation, 1 mark for BMI categorization, 1 mark for function calls, 1 mark for formatted output.

Part C: Notebook Maintenance (5 marks)

- **Neatness and organisation of notebook (2 marks):** Check for clean, organized notebook.
- **Completion of all assigned classwork and homework (2 marks):** Verify all work is complete.
- **Proper headings, dates, and indexing (1 mark):** Ensure proper formatting throughout.

ICT Class 8 - Formative Assessment

2

Formative Assessment (FA-2) - Answer Key

- Class 8

Class: Class 8

Assessment Type: Formative Assessment (FA-2)

Total Marks: 30

Duration: 90 minutes

Topics Covered: Data Structures - Lists, Tuples, Dictionaries

Curated and developed by

Hoysala T, Computer Science Teacher

MDRS SC-32 Bahaddurghatta (Kogunde), Chitradurga Taluk & District - 577519

CC BY-SA 4.0 - Free to reuse with attribution - 2026

Answer Key - Grading Guidelines

Note to Teacher: This answer key provides expected answers and marking guidelines. Accept equivalent answers where appropriate. Partial marks may be awarded for incomplete but correct responses.

Part A: Written Concept Paper (20 Marks)

Section I: Multiple Choice Questions (5 x 1 = 5 marks)

Q.No	Answer	Explanation
1	c	<code>my_list = []</code> creates an empty list; <code>()</code> is tuple, <code>{}</code> is dict
2	b	List index access is $O(1)$ because lists are arrays in memory
3	c	Tuples are immutable in Python; lists, dicts, and sets are mutable
4	b	Adding a new key-value pair makes the dictionary length 3
5	c	The <code>append()</code> method adds an element to the end of a list

Section II: Short Answer Questions (5 x 2 = 10 marks)

- Differentiate between a list and a tuple. Give two use cases of each.**
 - Answer:** Lists are mutable (can be modified after creation) while tuples are immutable. Lists use `[]` brackets, tuples use `()`. Use cases for lists: dynamic collections that change (shopping cart, todo list). Use cases for tuples: fixed data (coordinates, RGB values, database records).
 - Marking:** 1 mark for differentiation, 1 mark for valid use cases.
- Write the output of the list operations.**
 - Answer:** `[20, 30, 40]` and `[50, 40, 30, 20, 10]`
 - Marking:** 1 mark for first output (slice `[1:4]`), 1 mark for second output (reverse).
- Explain the difference between `append()` and `extend()` with examples.**
 - Answer:** `append()` adds a single element to the end of a list (e.g., `list1.append([4,5])` adds the list `[4,5]` as one element). `extend()` adds each element from an iterable individually (e.g., `list1.extend([4,5])` adds 4 and 5 as separate elements).
 - Marking:** 1 mark for explaining `append` with example, 1 mark for explaining `extend` with example.
- Write a Python code snippet to iterate over all key-value pairs in a dictionary using the `items()` method.**
 - Answer:**

```
my_dict = {"name": "Alice", "age": 14, "class": "8A"}
for key, value in my_dict.items():
    print(f"{key}: {value}")
```
 - Marking:** 1 mark for correct `items()` usage, 1 mark for correct loop structure.
- What is a nested dictionary? Give one real-life example where it is useful.**

- **Answer:** A nested dictionary is a dictionary where one or more values are themselves dictionaries. Example: A student database where each student has personal details as a nested dictionary: `students = {"s1": {"name": "Alice", "marks": {"math": 90, "science": 85}}}`.
- **Marking:** 1 mark for definition, 1 mark for valid real-life example.

Section III: Long Answer Questions (5 x 1 = 5 marks)

1. Write a Python program that manages a student marks database using a dictionary of lists.

- **Answer:**

```
students = {
    "Alice": [85, 90, 78, 92, 88],
    "Bob": [70, 75, 80, 65, 72],
    "Charlie": [95, 92, 88, 90, 94]
}

# Calculate averages
averages = {}
for name, marks in students.items():
    avg = sum(marks) / len(marks)
    averages[name] = avg

# Find highest and lowest
highest = max(averages, key=averages.get)
lowest = min(averages, key=averages.get)

# Display formatted table
print(f'{"Name":<10} {"Marks":<25} {"Average":<10}')
print("-" * 45)
for name, marks in students.items():
    marks_str = ", ".join(str(m) for m in marks)
    print(f'{name:<10} {marks_str:<25} {averages[name]:<10.1f}')

print(f"\nHighest average: {highest} ({averages[highest]:.1f})")
print(f"Lowest average: {lowest} ({averages[lowest]:.1f})")
```

- **Marking:** 1 mark for dictionary structure, 1 mark for average calculation, 1 mark for finding extremes, 1 mark for formatted output, 1 mark for complete program.

Part B: Hands-on Practical Lab Task (5 marks)

List Operations Program:

- **Answer:**

```
# Create list of 10 integers
nums = []
print("Enter 10 integers:")
for i in range(10):
    nums.append(int(input(f"Number {i+1}: ")))
```

```

# Sort, reverse, find max/min
sorted_nums = sorted(nums)
reversed_nums = nums[::-1]
max_val = max(nums)
min_val = min(nums)

# Calculate sum and average without built-in functions
total = 0
count = 0
for num in nums:
    total += num
    count += 1
average = total / count

# Create tuple from sorted list
num_tuple = tuple(sorted_nums)

print(f"Original: {nums}")
print(f"Sorted: {sorted_nums}")
print(f"Reversed: {reversed_nums}")
print(f"Max: {max_val}, Min: {min_val}")
print(f"Sum: {total}, Average: {average}")
print(f"Tuple: {num_tuple}")

```

- **Marking:** 1 mark for list creation, 1 mark for list methods, 1 mark for manual sum/average, 1 mark for tuple creation, 1 mark for output.

Part C: Notebook Maintenance (5 marks)

- **Neatness and organisation of notebook (2 marks):** Check for clean, organized notebook.
- **Completion of all assigned classwork and homework (2 marks):** Verify all work is complete.
- **Proper headings, dates, and indexing (1 mark):** Ensure proper formatting throughout.

ICT Class 8 - Formative Assessment

3

Formative Assessment (FA-3) - Answer Key

- Class 8

Class: Class 8

Assessment Type: Formative Assessment (FA-3)

Total Marks: 30

Duration: 90 minutes

Topics Covered: SQL - Database Fundamentals

Curated and developed by

Hoysala T, Computer Science Teacher

MDRS SC-32 Bahaddurghatta (Kogunde), Chitradurga Taluk & District - 577519

CC BY-SA 4.0 - Free to reuse with attribution - 2026

Answer Key - Grading Guidelines

Note to Teacher: This answer key provides expected answers and marking guidelines. Accept equivalent answers where appropriate. Partial marks may be awarded for incomplete but correct responses.

Part A: Written Concept Paper (20 Marks)

Section I: Multiple Choice Questions (5 x 1 = 5 marks)

Q.No	Answer	Explanation
1	b	SELECT is used to retrieve data from a database
2	c	WHERE clause filters rows based on conditions
3	c	DELETE FROM students WHERE roll_no = 5 deletes only the row where roll_no = 5
4	c	VARCHAR is used for variable-length text strings like names
5	b	DROP TABLE removes the table structure and all data permanently

Section II: Short Answer Questions (5 x 2 = 10 marks)

1. Write the SQL command to create a table Books.

• **Answer:**

```
CREATE TABLE Books (
    book_id INT PRIMARY KEY,
    title VARCHAR(100),
    author VARCHAR(50),
    price DECIMAL(10, 2),
    stock INT
);
```

- **Marking:** 1 mark for correct CREATE TABLE syntax, 1 mark for all columns with data types.

2. Differentiate between DELETE, TRUNCATE, and DROP commands.

- **Answer:** DELETE removes specific rows with WHERE clause and can be rolled back. TRUNCATE removes all rows but keeps structure, faster than DELETE. DROP removes the entire table including structure.

- **Marking:** 1 mark for each correct differentiation point (3 points for 2 marks).

3. Write SQL queries for insert, update, and delete operations.

• **Answer:**

```
-- Insert 3 records
INSERT INTO Books VALUES (1, 'Python Basics', 'John Smith', 450.00, 25);
INSERT INTO Books VALUES (2, 'Web Development', 'Jane Doe', 550.00, 15);
INSERT INTO Books VALUES (3, 'Database Systems', 'Bob Wilson', 600.00, 10);

-- Update price where book_id = 2
UPDATE Books SET price = 499.00 WHERE book_id = 2;
```

```
-- Delete where stock = 0
DELETE FROM Books WHERE stock = 0;
```

- **Marking:** 1 mark for correct INSERT statements, 1 mark for correct UPDATE and DELETE.

4. What is a Primary Key? What are its constraints?

- **Answer:** A Primary Key uniquely identifies each record in a table. Constraints: Must be unique (no duplicate values), cannot be NULL, each table can have only one primary key.
- **Marking:** 1 mark for definition, 1 mark for constraints.

5. Write a SQL query to display books sorted by price descending, priced above ₹500.

- **Answer:**

```
SELECT * FROM Books
WHERE price > 500
ORDER BY price DESC;
```

- **Marking:** 1 mark for WHERE clause, 1 mark for ORDER BY clause.

Section III: Long Answer Questions (5 x 1 = 5 marks)

1. Write SQL queries for the Students table.

- **Answer:**

```
-- Display students who scored more than 80 marks
SELECT * FROM Students WHERE marks > 80;
```

```
-- Count students in each section
SELECT section, COUNT(*) as student_count
FROM Students
GROUP BY section;
```

```
-- Find max, min, and average marks
SELECT MAX(marks) as maximum, MIN(marks) as minimum, AVG(marks) as average
FROM Students;
```

```
-- Display students sorted by marks descending
SELECT * FROM Students ORDER BY marks DESC;
```

```
-- Show only top 5 students
SELECT * FROM Students ORDER BY marks DESC LIMIT 5;
```

- **Marking:** 1 mark for each correct query (5 queries).

Part B: Hands-on Practical Lab Task (5 marks)

SQLite Database Program:

- **Answer:**

```
import sqlite3

# Create in-memory database
conn = sqlite3.connect(':memory:')
cursor = conn.cursor()
```

```

# Create Employees table
cursor.execute('''
    CREATE TABLE Employees (
        emp_id INTEGER PRIMARY KEY,
        name TEXT NOT NULL,
        department TEXT,
        salary REAL,
        joining_date DATE
    )
''')

# Insert 8 records
employees = [
    (1, 'Alice', 'IT', 75000, '2021-03-15'),
    (2, 'Bob', 'HR', 65000, '2020-07-20'),
    (3, 'Charlie', 'IT', 80000, '2019-01-10'),
    (4, 'Diana', 'Finance', 70000, '2022-05-25'),
    (5, 'Eve', 'HR', 68000, '2021-11-30'),
    (6, 'Frank', 'IT', 72000, '2020-09-05'),
    (7, 'Grace', 'Finance', 85000, '2018-12-01'),
    (8, 'Henry', 'Marketing', 60000, '2023-02-14')
]
cursor.executemany('INSERT INTO Employees VALUES (?, ?, ?, ?, ?)', employees)

# Query 1: SELECT with WHERE
print("IT Department Employees:")
cursor.execute("SELECT * FROM Employees WHERE department = 'IT'")
for row in cursor.fetchall():
    print(row)

# Query 2: ORDER BY
print("\nEmployees sorted by salary:")
cursor.execute("SELECT name, salary FROM Employees ORDER BY salary DESC")
for row in cursor.fetchall():
    print(row)

# Query 3: GROUP BY
print("\nEmployees per department:")
cursor.execute("SELECT department, COUNT(*) FROM Employees GROUP BY department")
for row in cursor.fetchall():
    print(row)

# Query 4: Aggregate functions
print("\nSalary statistics:")
cursor.execute("SELECT AVG(salary), MAX(salary), MIN(salary) FROM Employees")
for row in cursor.fetchall():
    print(f"Average: {row[0]:.2f}, Max: {row[1]}, Min: {row[2]}")

# Query 5: UPDATE
cursor.execute("UPDATE Employees SET salary = salary * 1.1 WHERE department = 'IT'")
conn.commit()

```

conn.close()

- **Marking:** 1 mark for database creation, 1 mark for table creation, 1 mark for inserting records, 1 mark for queries, 1 mark for output formatting.

Part C: Notebook Maintenance (5 marks)

- **Neatness and organisation of notebook (2 marks):** Check for clean, organized notebook.
- **Completion of all assigned classwork and homework (2 marks):** Verify all work is complete.
- **Proper headings, dates, and indexing (1 mark):** Ensure proper formatting throughout.

ICT Class 8 - Formative Assessment

4

Formative Assessment (FA-4) - Answer Key

- Class 8

Class: Class 8

Assessment Type: Formative Assessment (FA-4)

Total Marks: 30

Duration: 90 minutes

Topics Covered: HTML and CSS

Curated and developed by

Hoysala T, Computer Science Teacher

MDRS SC-32 Bahaddurghatta (Kogunde), Chitradurga Taluk & District - 577519

CC BY-SA 4.0 - Free to reuse with attribution - 2026

Answer Key - Grading Guidelines

Note to Teacher: This answer key provides expected answers and marking guidelines. Accept equivalent answers where appropriate. Partial marks may be awarded for incomplete but correct responses.

Part A: Written Concept Paper (20 Marks)

Section I: Multiple Choice Questions (5 x 1 = 5 marks)

Q.No	Answer	Explanation
1	b	<a> tag is used to create hyperlinks in HTML
2	c	color property changes text colour in CSS
3	c	 tag is used to display images in HTML
4	b	.header targets elements with class name "header"
5	c	<div> is a block-level element; others are inline

Section II: Short Answer Questions (5 x 2 = 10 marks)

1. Differentiate between HTML and CSS.

- **Answer:** HTML (HyperText Markup Language) defines the structure and content of web pages. CSS (Cascading Style Sheets) controls the presentation and styling of HTML elements. HTML uses tags like <h1>, <p>, while CSS uses selectors and properties like color, margin.
- **Marking:** 1 mark for HTML role, 1 mark for CSS role.

2. Write the HTML code to create a table with 3 columns and 4 rows.

- **Answer:**

```
<table border="1">
  <tr>
    <th>Name</th>
    <th>Age</th>
    <th>Grade</th>
  </tr>
  <tr>
    <td>Alice</td>
    <td>13</td>
    <td>A</td>
  </tr>
  <tr>
    <td>Bob</td>
    <td>14</td>
    <td>B+</td>
  </tr>
  <tr>
    <td>Charlie</td>
```

```

        <td>13</td>
        <td>A-</td>
    </tr>
</table>

```

- **Marking:** 1 mark for correct table structure, 1 mark for all rows and columns.

3. Explain the difference between inline, internal, and external CSS.

- **Answer:** Inline CSS uses `style` attribute directly on elements (e.g., `<p style="color: red;">`). Internal CSS uses `<style>` tag in the `<head>` section. External CSS uses a separate `.css` file linked via `<link>` tag. External CSS is best for maintainability.
- **Marking:** 1 mark for explaining each type, 1 mark for examples.

4. Explain the CSS box model with a labelled diagram.

- **Answer:** The box model consists of: Content (actual text/image), Padding (space between content and border), Border (surrounds padding), Margin (space outside border). Diagram should show these four layers with labels.
- **Marking:** 1 mark for explaining components, 1 mark for diagram.

5. Write CSS rules for background colour, h2 border, and paragraph styling.

- **Answer:**

```

body {
    background-color: #f0f0f0;
}
h2 {
    border: 2px solid blue;
}
p {
    font-size: 16px;
    line-height: 1.5;
}

```

- **Marking:** 1 mark for body background, 1 mark for h2 border and p styling.

Section III: Long Answer Questions (5 x 1 = 5 marks)

1. Create a complete HTML page for a School Library Portal.

- **Answer:** Complete HTML page with:
 - Navigation bar with 4 links using `<nav>` and `<a>` tags
 - Header with school name using `<header>` and `<h1>`
 - Table with 6 books using `<table>`, `<tr>`, `<td>`
 - Form with input fields using `<form>`, `<input>`
 - CSS styling with colour scheme, hover effects, alternating row colours, responsive layout
- **Marking:** 1 mark for navigation, 1 mark for header, 1 mark for table, 1 mark for form, 1 mark for CSS styling.

Part B: Hands-on Practical Lab Task (5 marks)

Personal Portfolio HTML Page:

- **Answer:**


```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>My Portfolio</title>
  <link rel="stylesheet" href="styles.css">
</head>
<body>
  <header>
    <h1>John Doe</h1>
    <p>Web Developer & Designer</p>
  </header>

  <section class="about">
    <h2>About Me</h2>
    
    <p>I am a passionate web developer with skills in HTML, CSS, and JavaScript.</p>
  </section>

  <section class="skills">
    <h2>Skills</h2>
    <div class="skills-grid">
      <div class="skill">HTML</div>
      <div class="skill">CSS</div>
      <div class="skill">JavaScript</div>
      <div class="skill">Python</div>
      <div class="skill">SQL</div>
    </div>
  </section>

  <section class="contact">
    <h2>Contact Me</h2>
    <form>
      <input type="text" placeholder="Name" required>
      <input type="email" placeholder="Email" required>
      <textarea placeholder="Message" required></textarea>
      <button type="submit">Submit</button>
    </form>
  </section>
</body>
</html>

```

- **Marking:** 1 mark for header with name/tagline, 1 mark for about section with image, 1 mark for skills grid, 1 mark for contact form, 1 mark for responsive CSS.

Part C: Notebook Maintenance (5 marks)

- **Neatness and organisation of notebook (2 marks):** Check for clean, organized notebook.
- **Completion of all assigned classwork and homework (2 marks):** Verify all work is complete.

- **Proper headings, dates, and indexing (1 mark):** Ensure proper formatting throughout.

ICT Class 8 - Practical Lab Exam 1

Practical Lab Exam 1 - Answer Key

- Class 8

Class: Class 8

Assessment Type: Practical Lab Exam 1

Total Marks: 20

Duration: 45 minutes

Topics Covered: Python Functions and Data Structures

Curated and developed by

Hoysala T, Computer Science Teacher

MDRS SC-32 Bahaddurghatta (Kogunde), Chitradurga Taluk & District - 577519

CC BY-SA 4.0 - Free to reuse with attribution - 2026

Answer Key - Grading Guidelines

Note to Teacher: This answer key provides expected answers and marking guidelines. Accept equivalent answers where appropriate. Partial marks may be awarded for incomplete but correct responses.

Lab Tasks (15 marks)

Task 1: Function-Based Calculator (5 marks)

Complete Python Program:

• **Answer:**

```
def add(a, b):
    """Return sum of two numbers."""
    return a + b

def subtract(a, b):
    """Return difference of two numbers."""
    return a - b

def multiply(a, b):
    """Return product of two numbers."""
    return a * b

def divide(a, b):
    """Return division of two numbers with zero check."""
    if b == 0:
        return "Error: Division by zero"
    return a / b

def main():
    while True:
        print("\n=== Calculator Menu ===")
        print("1. Add")
        print("2. Subtract")
        print("3. Multiply")
        print("4. Divide")
        print("5. Exit")

        choice = input("\nEnter your choice (1-5): ")

        if choice == '5':
            print("Thank you for using the calculator!")
            break

        if choice not in ['1', '2', '3', '4']:
            print("Invalid choice! Please try again.")
            continue
```

```

try:
    num1 = float(input("Enter first number: "))
    num2 = float(input("Enter second number: "))
except ValueError:
    print("Invalid input! Please enter numbers.")
    continue

if choice == '1':
    result = add(num1, num2)
    print(f"{num1} + {num2} = {result}")
elif choice == '2':
    result = subtract(num1, num2)
    print(f"{num1} - {num2} = {result}")
elif choice == '3':
    result = multiply(num1, num2)
    print(f"{num1} × {num2} = {result}")
elif choice == '4':
    result = divide(num1, num2)
    print(f"{num1} ÷ {num2} = {result}")

if __name__ == "__main__":
    main()

```

- **Marking:** 1 mark for function definitions, 1 mark for division by zero handling, 1 mark for menu-driven approach, 1 mark for input validation, 1 mark for formatted output.

Task 2: List Operations Manager (5 marks)

Complete Python Program:

- **Answer:**

```

def display_list(lst):
    """Display the current list."""
    if not lst:
        print("List is empty")
    else:
        print("Current list:", lst)

def main():
    names = ["Alice", "Bob", "Charlie", "Diana", "Eve",
             "Frank", "Grace", "Henry", "Ivy", "Jack",
             "Kate", "Leo", "Mia", "Noah", "Olivia"]

    while True:
        print("\n=== List Operations Manager ===")
        print("1. Add a name")
        print("2. Remove a name")
        print("3. Search for a name")
        print("4. Sort the list")
        print("5. Reverse the list")
        print("6. Display all names")
        print("7. Exit")

```

```

choice = input("\nEnter your choice (1-7): ")

if choice == '7':
    print("Goodbye!")
    break

if choice == '1':
    name = input("Enter name to add: ")
    names.append(name)
    print(f"'{name}' added successfully!")
    display_list(names)

elif choice == '2':
    name = input("Enter name to remove: ")
    if name in names:
        names.remove(name)
        print(f"'{name}' removed successfully!")
    else:
        print(f"Error: '{name}' not found in the list!")
    display_list(names)

elif choice == '3':
    name = input("Enter name to search: ")
    if name in names:
        print(f"'{name}' found at index {names.index(name)}")
    else:
        print(f"'{name}' not found in the list")

elif choice == '4':
    names.sort()
    print("List sorted alphabetically!")
    display_list(names)

elif choice == '5':
    names.reverse()
    print("List reversed!")
    display_list(names)

elif choice == '6':
    display_list(names)

else:
    print("Invalid choice! Please try again.")

if __name__ == "__main__":
    main()

```

- **Marking:** 1 mark for list creation, 1 mark for menu implementation, 1 mark for list methods, 1 mark for input validation, 1 mark for display after operations.

Task 3: Dictionary-Based Marks Manager (5 marks)

Complete Python Program:

• Answer:

```
def calculate_average(marks):
    """Calculate average of marks."""
    return sum(marks.values()) / len(marks)

def find_topper(students):
    """Find student with highest average."""
    topper = None
    highest_avg = -1
    for name, marks in students.items():
        avg = calculate_average(marks)
        if avg > highest_avg:
            highest_avg = avg
            topper = name
    return topper, highest_avg

def display_students(students):
    """Display all students and their marks."""
    print(f"\n{'Name':<10} {'Math':<8} {'Science':<10} {'English':<10} {'SST':<8}"
          {'Average':<10}")
    print("-" * 56)
    for name, marks in students.items():
        avg = calculate_average(marks)
        print(f"{name:<10} {marks['Math']:<8} {marks['Science']:<10}"
              {marks['English']:<10} {marks['SST']:<8} {avg:<10.1f}")

def main():
    students = {
        "Alice": {"Math": 85, "Science": 90, "English": 78, "SST": 88},
        "Bob": {"Math": 70, "Science": 75, "English": 80, "SST": 65},
        "Charlie": {"Math": 95, "Science": 92, "English": 88, "SST": 90},
        "Diana": {"Math": 82, "Science": 78, "English": 85, "SST": 80},
        "Eve": {"Math": 88, "Science": 85, "English": 92, "SST": 78},
        "Frank": {"Math": 72, "Science": 80, "English": 75, "SST": 82},
        "Grace": {"Math": 90, "Science": 88, "English": 85, "SST": 92},
        "Henry": {"Math": 78, "Science": 82, "English": 80, "SST": 75}
    }

    while True:
        print("\n=== Marks Manager ===")
        print("1. Add new student")
        print("2. Update marks")
        print("3. Display all students")
        print("4. Calculate averages")
        print("5. Find topper")
        print("6. Exit")

        choice = input("\nEnter your choice (1-6): ")
```

```

if choice == '6':
    print("Goodbye!")
    break

if choice == '1':
    name = input("Enter student name: ")
    if name in students:
        print("Student already exists!")
    else:
        students[name] = {}
        for subject in ["Math", "Science", "English", "SST"]:
            marks = int(input(f"Enter {subject} marks: "))
            students[name][subject] = marks
        print(f"Student '{name}' added successfully!")

elif choice == '2':
    name = input("Enter student name: ")
    if name not in students:
        print("Student not found!")
    else:
        subject = input("Enter subject (Math/Science/English/SST): ")
        if subject not in ["Math", "Science", "English", "SST"]:
            print("Invalid subject!")
        else:
            marks = int(input(f"Enter new {subject} marks: "))
            students[name][subject] = marks
            print("Marks updated successfully!")

elif choice == '3':
    display_students(students)

elif choice == '4':
    display_students(students)

elif choice == '5':
    topper, avg = find_topper(students)
    print(f"\nTopper: {topper} with average {avg:.1f}")

else:
    print("Invalid choice! Please try again.")

if __name__ == "__main__":
    main()

```

- **Marking:** 1 mark for dictionary structure, 1 mark for add/update functions, 1 mark for average calculation, 1 mark for topper finding, 1 mark for formatted output.

Viva Voce (5 marks)

1. What is the difference between a function definition and a function call?

- **Answer:** Function definition is where you write the function code using `def` keyword. Function call is where you execute the function by using its name followed by parentheses.
 - **Marking:** 1 mark for correct explanation.
2. **Can a function have both default parameters and variable-length arguments?**
- **Answer:** Yes. Example: `def func(a, b=5, *args):` where `a` is required, `b` has default value, and `args` accepts variable arguments.
 - **Marking:** 1 mark for correct answer with example.
3. **Why are dictionaries preferred over lists for key-value pairs?**
- **Answer:** Dictionaries provide $O(1)$ average time complexity for lookups using keys. Lists require $O(n)$ time to search for values. Dictionaries also provide meaningful key names for better code readability.
 - **Marking:** 1 mark for explaining efficiency and readability.
4. **What is the difference between `remove()` and `pop()` for lists?**
- **Answer:** `remove(x)` searches for value `x` and removes first occurrence, returns nothing. `pop(index)` removes and returns element at specified index, or last element if no index given.
 - **Marking:** 1 mark for explaining both methods.
5. **How does Python handle memory with deep copy vs shallow copy?**
- **Answer:** Shallow copy creates new object but references same inner objects (changes affect original). Deep copy creates new object and recursively copies all inner objects (completely independent). Use `copy.deepcopy()` for nested structures.
 - **Marking:** 1 mark for explaining memory handling difference.

ICT Class 8 - Practical Lab Exam 2

Practical Lab Exam 2 - Answer Key

- Class 8

Class: Class 8

Assessment Type: Practical Lab Exam 2

Total Marks: 20

Duration: 45 minutes

Topics Covered: SQL with Python and Web Development

Curated and developed by

Hoysala T, Computer Science Teacher

MDRS SC-32 Bahaddurghatta (Kogunde), Chitradurga Taluk & District - 577519

CC BY-SA 4.0 - Free to reuse with attribution - 2026

Answer Key - Grading Guidelines

Note to Teacher: This answer key provides expected answers and marking guidelines. Accept equivalent answers where appropriate. Partial marks may be awarded for incomplete but correct responses.

Lab Tasks (15 marks)

Task 1: SQL Database Operations with Python (5 marks)

Complete Python Program:

• **Answer:**

```
import sqlite3

# Create database and table
conn = sqlite3.connect('school.db')
cursor = conn.cursor()

cursor.execute('''
    CREATE TABLE IF NOT EXISTS Students (
        roll_no INTEGER PRIMARY KEY,
        name TEXT NOT NULL,
        class TEXT,
        section TEXT,
        marks INTEGER
    )
''')

# Insert 8 records
students = [
    (1, 'Alice', '8', 'A', 85),
    (2, 'Bob', '8', 'B', 72),
    (3, 'Charlie', '8', 'A', 90),
    (4, 'Diana', '8', 'C', 78),
    (5, 'Eve', '8', 'B', 88),
    (6, 'Frank', '8', 'A', 65),
    (7, 'Grace', '8', 'C', 92),
    (8, 'Henry', '8', 'B', 70)
]

cursor.executemany('INSERT OR IGNORE INTO Students VALUES (?, ?, ?, ?, ?)', students)
conn.commit()

# 1. SELECT with WHERE clause
print("Students with marks above 80:")
cursor.execute("SELECT * FROM Students WHERE marks > 80")
for row in cursor.fetchall():
    print(row)
```

```
# 2. SELECT with ORDER BY
print("\nStudents sorted by marks (descending):")
cursor.execute("SELECT name, marks FROM Students ORDER BY marks DESC")
for row in cursor.fetchall():
    print(f"{row[0]}: {row[1]}")

# 3. UPDATE operation
cursor.execute("UPDATE Students SET marks = marks + 5 WHERE roll_no = 2")
conn.commit()
print("\nUpdated Bob's marks (+5)")

# 4. DELETE operation
cursor.execute("DELETE FROM Students WHERE roll_no = 6")
conn.commit()
print("Deleted Frank's record")

# 5. COUNT and AVG aggregate queries
print("\nTotal students:", cursor.execute("SELECT COUNT(*) FROM Students").fetchone()[0])
print("Average marks:", cursor.execute("SELECT AVG(marks) FROM Students").fetchone()[0])

conn.close()
```

- **Marking:** 1 mark for database and table creation, 1 mark for inserting records, 1 mark for SELECT queries, 1 mark for UPDATE/DELETE, 1 mark for aggregate queries.

Task 2: HTML and CSS Web Page (5 marks)

HTML File (student_portal.html):

- **Answer:**

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Student Portal</title>
  <link rel="stylesheet" href="styles.css">
</head>
<body>
  <header>
    <div class="header-content">
      <h1>our school Student Portal</h1>
      <nav>
        <a href="#">Home</a>
        <a href="#">Students</a>
        <a href="#">Results</a>
        <a href="#">Contact</a>
      </nav>
    </div>
  </header>
```

```

<main>
  <h2>Student Records</h2>
  <table>
    <thead>
      <tr>
        <th>Name</th>
        <th>Class</th>
        <th>Section</th>
        <th>Marks</th>
        <th>Grade</th>
      </tr>
    </thead>
    <tbody>
      <tr>
        <td>Alice</td>
        <td>8</td>
        <td>A</td>
        <td>85</td>
        <td>A</td>
      </tr>
      <tr>
        <td>Bob</td>
        <td>8</td>
        <td>B</td>
        <td>72</td>
        <td>B+</td>
      </tr>
      <tr>
        <td>Charlie</td>
        <td>8</td>
        <td>A</td>
        <td>90</td>
        <td>A+</td>
      </tr>
      <tr>
        <td>Diana</td>
        <td>8</td>
        <td>C</td>
        <td>78</td>
        <td>B+</td>
      </tr>
      <tr>
        <td>Eve</td>
        <td>8</td>
        <td>B</td>
        <td>88</td>
        <td>A</td>
      </tr>
      <tr>
        <td>Grace</td>
        <td>8</td>

```

```

        <td>C</td>
        <td>92</td>
        <td>A+</td>
    </tr>
</tbody>
</table>
</main>

<footer>
    <p>&copy; 2024 our school Student Portal. All rights reserved.</p>
</footer>
</body>
</html>

```

CSS File (styles.css):

- **Answer:**

```

* {
    margin: 0;
    padding: 0;
    box-sizing: border-box;
}

body {
    font-family: Arial, sans-serif;
    line-height: 1.6;
}

header {
    background: linear-gradient(135deg, #1a237e, #283593);
    color: white;
    padding: 1rem 2rem;
}

.header-content {
    display: flex;
    justify-content: space-between;
    align-items: center;
    max-width: 1200px;
    margin: 0 auto;
}

nav a {
    color: white;
    text-decoration: none;
    margin-left: 20px;
    padding: 8px 16px;
    border-radius: 4px;
    transition: background 0.3s;
}

nav a:hover {

```

```

    background: rgba(255, 255, 255, 0.2);
}

main {
    max-width: 1200px;
    margin: 2rem auto;
    padding: 0 2rem;
}

h2 {
    margin-bottom: 1rem;
    color: #1a237e;
}

table {
    width: 100%;
    border-collapse: collapse;
    box-shadow: 0 2px 10px rgba(0,0,0,0.1);
}

th, td {
    padding: 12px 15px;
    text-align: left;
    border-bottom: 1px solid #ddd;
}

th {
    background-color: #1a237e;
    color: white;
}

tbody tr:nth-child(even) {
    background-color: #f5f5f5;
}

tbody tr:hover {
    background-color: #e3f2fd;
    transition: background 0.3s;
}

footer {
    background: #333;
    color: white;
    text-align: center;
    padding: 1rem;
    margin-top: 2rem;
}

@media (max-width: 768px) {
    .header-content {
        flex-direction: column;
    }
}

```

```

    text-align: center;
}

nav {
    margin-top: 1rem;
}

nav a {
    margin: 0 10px;
}

table {
    font-size: 14px;
}

th, td {
    padding: 8px 10px;
}
}

```

- **Marking:** 1 mark for HTML structure, 1 mark for table with data, 1 mark for CSS styling, 1 mark for alternating rows and hover, 1 mark for responsive design.

Task 3: SQL Query Challenge (5 marks)

Complete Python Program:

- **Answer:**

```

import sqlite3
from datetime import datetime

conn = sqlite3.connect(':memory:')
cursor = conn.cursor()

# Create tables with Foreign Key
cursor.execute('''
    CREATE TABLE Employees (
        emp_id INTEGER PRIMARY KEY,
        name TEXT NOT NULL,
        department TEXT,
        salary REAL,
        joining_date DATE
    )
''')

cursor.execute('''
    CREATE TABLE Projects (
        project_id INTEGER PRIMARY KEY,
        project_name TEXT,
        emp_id INTEGER,
        start_date DATE,
        end_date DATE,
        FOREIGN KEY (emp_id) REFERENCES Employees(emp_id)
    )
''')

```



```

    )
'''

# Insert employees
employees = [
    (1, 'Alice', 'IT', 75000, '2021-03-15'),
    (2, 'Bob', 'HR', 65000, '2020-07-20'),
    (3, 'Charlie', 'IT', 80000, '2019-01-10'),
    (4, 'Diana', 'Finance', 70000, '2022-05-25'),
    (5, 'Eve', 'HR', 68000, '2021-11-30'),
    (6, 'Frank', 'IT', 72000, '2020-09-05')
]
cursor.executemany('INSERT INTO Employees VALUES (?, ?, ?, ?, ?)', employees)

# Insert projects
projects = [
    (1, 'Website Redesign', 1, '2023-01-01', '2023-06-30'),
    (2, 'Mobile App', 1, '2023-03-15', None),
    (3, 'HR Portal', 2, '2023-02-01', '2023-08-31'),
    (4, 'Data Analytics', 3, '2023-04-01', None),
    (5, 'Finance System', 4, '2023-05-15', '2023-12-31'),
    (6, 'Employee Training', 2, '2023-06-01', None),
    (7, 'Security Audit', 3, '2023-07-01', None),
    (8, 'Budget Planning', 5, '2023-08-01', '2024-01-31'),
    (9, 'IT Support', 6, '2023-09-01', None),
    (10, 'Cloud Migration', 1, '2023-10-01', None)
]
cursor.executemany('INSERT INTO Projects VALUES (?, ?, ?, ?, ?)', projects)
conn.commit()

# 1. Employees earning more than average salary
print("1. Employees earning more than average salary:")
cursor.execute('''
    SELECT name, salary FROM Employees
    WHERE salary > (SELECT AVG(salary) FROM Employees)
''')
for row in cursor.fetchall():
    print(f"    {row[0]}: ₹{row[1]}")

# 2. Departments with more than 2 employees
print("\n2. Departments with more than 2 employees:")
cursor.execute('''
    SELECT department, COUNT(*) as emp_count
    FROM Employees
    GROUP BY department
    HAVING COUNT(*) > 2
''')
for row in cursor.fetchall():
    print(f"    {row[0]}: {row[1]} employees")

# 3. Employees working on most projects

```

```

print("\n3. Employees working on most projects:")
cursor.execute('''
    SELECT e.name, COUNT(p.project_id) as project_count
    FROM Employees e
    JOIN Projects p ON e.emp_id = p.emp_id
    GROUP BY e.emp_id
    ORDER BY project_count DESC
    LIMIT 1
''')
for row in cursor.fetchall():
    print(f"    {row[0]}: {row[1]} projects")

# 4. Total salary expenditure per department
print("\n4. Total salary expenditure per department:")
cursor.execute('''
    SELECT department, SUM(salary) as total_salary
    FROM Employees
    GROUP BY department
    ORDER BY total_salary DESC
''')
for row in cursor.fetchall():
    print(f"    {row[0]}: ₹{row[1]}")

# 5. Ongoing projects (no end date or end date > today)
print("\n5. Ongoing projects:")
today = datetime.now().strftime('%Y-%m-%d')
cursor.execute('''
    SELECT p.project_name, e.name as emp_name
    FROM Projects p
    JOIN Employees e ON p.emp_id = e.emp_id
    WHERE p.end_date IS NULL OR p.end_date > ?
''', (today,))
for row in cursor.fetchall():
    print(f"    {row[0]} (Assigned to: {row[1]})")

conn.close()

```

- **Marking:** 1 mark for table creation with foreign key, 1 mark for data insertion, 1 mark for subquery and JOIN queries, 1 mark for aggregate queries, 1 mark for ongoing projects query.

Viva Voce (5 marks)

1. What is the difference between INNER JOIN and LEFT JOIN?

- **Answer:** INNER JOIN returns only matching records from both tables. LEFT JOIN returns all records from left table and matching records from right table (NULL for non-matching).
Example: SELECT * FROM A INNER JOIN B ON A.id = B.a_id VS SELECT * FROM A LEFT JOIN B ON A.id = B.a_id.
- **Marking:** 1 mark for explaining both joins with examples.

2. How does CSS specificity work?

- **Answer:** Specificity determines which CSS rule applies when multiple rules target same element. Priority: Inline styles (highest) > Internal/External styles > Browser default (lowest). Within same level, ID selectors > class selectors > element selectors.
- **Marking:** 1 mark for explaining specificity levels.

3. **What is the purpose of a Foreign Key in SQL?**

- **Answer:** Foreign Key creates a link between two tables by referencing the primary key of another table. It maintains referential integrity by preventing invalid data entries and enabling relationships between related data.
- **Marking:** 1 mark for explaining purpose and data integrity.

4. **What is the difference between WHERE and HAVING in SQL?**

- **Answer:** WHERE filters rows before grouping (cannot use aggregate functions). HAVING filters groups after GROUP BY (can use aggregate functions). Example: WHERE salary > 50000 vs HAVING AVG(salary) > 40000.
- **Marking:** 1 mark for explaining both clauses with examples.

5. **Explain difference between responsive and adaptive design.**

- **Answer:** Responsive design uses fluid grids and media queries to adapt to any screen size continuously. Adaptive design uses fixed layouts for specific screen sizes and switches between them. Responsive is more flexible; adaptive can be more performant.
- **Marking:** 1 mark for explaining both approaches.

ICT Class 8 - Summative Assessment 1

Summative Assessment (SA-1) - Answer Key

- Class 8

Class: Class 8

Assessment Type: Summative Assessment (SA-1)

Total Marks: 40

Duration: 2 hours

Topics Covered: Functions, Data Structures, SQL, HTML/CSS

Curated and developed by

Hoysala T, Computer Science Teacher

MDRS SC-32 Bahaddurghatta (Kogunde), Chitradurga Taluk & District - 577519

CC BY-SA 4.0 - Free to reuse with attribution - 2026

Answer Key - Grading Guidelines

Note to Teacher: This answer key provides expected answers and marking guidelines. Accept equivalent answers where appropriate. Partial marks may be awarded for incomplete but correct responses.

Part A: MCQ (10 x 1 = 10 marks)

Q.No	Answer	Explanation
1	b	def keyword defines a function in Python
2	b	len([1, [2, 3], 4]) returns 3 (three top-level elements)
3	c	TRUNCATE deletes all rows but keeps table structure
4	a	<tr> tag creates a table row in HTML
5	c	ID selector #main has highest specificity
6	b	Output: [1] then [1, 2] due to mutable default argument
7	b	sorted() returns a new sorted list without modifying original
8	c	SELECT DISTINCT * FROM table; displays unique records
9	b	padding adds space between element border and content
10	c	List is mutable; tuple, string, integer are immutable

Part B: Short Answer (8 x 2 = 16 marks)

1. Write a Python function reverse_string(s).

• **Answer:**

```
def reverse_string(s):
    """Return the reversed version of string s."""
    if not s:
        return ""
    return s[::-1]

# Test cases
print(reverse_string("hello"))    # Output: "olleh"
print(reverse_string(""))        # Output: ""
print(reverse_string("Python"))  # Output: "nohtyP"
```

• **Marking:** 1 mark for function definition, 1 mark for empty string handling.

2. Write the output and explain line by line.

• **Answer:** Output: 1, 120, 6. Line 1: factorial(0) returns 1 (base case). Line 2: factorial(5) = 5 × 4 × 3 × 2 × 1 = 120. Line 3: factorial(3) = 3 × 2 × 1 = 6.

• **Marking:** 1 mark for correct output, 1 mark for line-by-line explanation.

3. Differentiate between list.append() and list.extend().

- **Answer:** `append()` adds a single element to the end (e.g., `list1.append([4,5])` adds `[4,5]` as one element). `extend()` adds each element individually (e.g., `list1.extend([4,5])` adds 4 and 5 separately).
- **Marking:** 1 mark for explaining `append`, 1 mark for explaining `extend`.

4. Write a Python program to count word frequency.

- **Answer:**

```
sentence = "the cat sat on the mat the cat"
words = sentence.split()
freq = {}
for word in words:
    freq[word] = freq.get(word, 0) + 1

for word, count in sorted(freq.items(), key=lambda x: x[1], reverse=True):
    print(f"'{word}': {count}")
```

- **Marking:** 1 mark for dictionary creation, 1 mark for frequency counting and display.

5. Write SQL queries for Employees table.

- **Answer:**

```
-- Find employees with salary between 30000 and 50000
SELECT * FROM Employees WHERE salary BETWEEN 30000 AND 50000;

-- Count employees per department
SELECT department, COUNT(*) FROM Employees GROUP BY department;

-- Find employee with maximum salary
SELECT * FROM Employees WHERE salary = (SELECT MAX(salary) FROM Employees);

-- Delete employees who joined before 2020
DELETE FROM Employees WHERE joining_date < '2020-01-01';
```

- **Marking:** 1 mark for each correct query (4 queries).

6. Differentiate between DELETE and TRUNCATE.

- **Answer:** DELETE: Removes specific rows, can use WHERE clause, can be rolled back, logs each row deletion. TRUNCATE: Removes all rows, faster than DELETE, cannot use WHERE clause, cannot be rolled back.
- **Marking:** 1 mark for three points about DELETE, 1 mark for three points about TRUNCATE.

7. Write HTML code for a form.

- **Answer:**

```
<form>
  <label for="name">Name:</label>
  <input type="text" id="name" name="name" required><br><br>

  <label for="email">Email:</label>
  <input type="email" id="email" name="email" required><br><br>

  <label for="dob">Date of Birth:</label>
  <input type="date" id="dob" name="dob" required><br><br>
```

```
<label>Gender:</label>
<input type="radio" id="male" name="gender" value="male">
<label for="male">Male</label>
<input type="radio" id="female" name="gender" value="female">
<label for="female">Female</label><br><br>
```

```
<input type="submit" value="Submit">
```

```
</form>
```

- **Marking:** 1 mark for form structure, 1 mark for all input types.

8. Write CSS rules for navigation, links, and media query.

- **Answer:**

```
/* Responsive navigation bar with flexbox */
```

```
nav {
  display: flex;
  justify-content: space-around;
  background-color: #333;
  padding: 10px;
}
```

```
nav a {
  color: white;
  text-decoration: none;
  padding: 10px 20px;
}
```

```
/* Hover underline effect */
nav a:hover {
  text-decoration: underline;
}
```

```
/* Media query for screens below 768px */
```

```
@media (max-width: 768px) {
  nav {
    flex-direction: column;
    align-items: center;
  }
```

```
  nav a {
    padding: 5px;
  }
}
```

- **Marking:** 1 mark for flexbox navigation, 1 mark for media query.

Part C: Long Answer (4 x 3 = 12 marks)

1. Write a complete Python program for a shopping cart.

- **Answer:** Complete program with functions for adding items, removing items, calculating total with GST, and displaying formatted receipt. Should include proper error handling and formatted output.

- **Marking:** 1 mark for add/remove functions, 1 mark for GST calculation, 1 mark for formatted receipt.
2. **Write SQL queries for School database.**
 - **Answer:** Complex queries using JOINS, subqueries, and aggregate functions to find students scoring above 80, calculate percentages, find highest average subject, and rank students.
 - **Marking:** 1 mark for each correct query approach (4 queries).
 3. **Create HTML page for Student Report Card.**
 - **Answer:** Complete HTML with header, styled table, summary statistics, CSS for professional formatting, and print-friendly layout.
 - **Marking:** 1 mark for HTML structure, 1 mark for table styling, 1 mark for print-friendly CSS.
 4. **Write a Python Quiz Application.**
 - **Answer:** Program with dictionary for questions, input validation, score tracking, and results display with percentage and grade.
 - **Marking:** 1 mark for question storage, 1 mark for input handling, 1 mark for score calculation.

Part D: Application Based (2 x 1 = 2 marks)

1. **Identify the bug in the add_item function.**
 - **Answer:** Bug: Using mutable default argument `cart=[]`. The list persists between function calls. Corrected version: Use `None` as default and create new list inside function.
 - **Marking:** 1 mark for identifying bug, 1 mark for corrected version.
2. **Explain the SQL query step by step.**
 - **Answer:** Query selects department, counts employees, calculates average salary for employees who joined after 2020, groups by department, filters groups with average salary > 40000, sorts by average salary descending.
 - **Marking:** 1 mark for explaining each clause's role.

ICT Class 8 - Summative Assessment 2

Summative Assessment (SA-2) - Answer Key

- Class 8

Class: Class 8

Assessment Type: Summative Assessment (SA-2)

Total Marks: 40

Duration: 2 hours

Topics Covered: Algorithms, Emerging Technologies, Cyber Safety, Computational Thinking

Curated and developed by

Hoysala T, Computer Science Teacher

MDRS SC-32 Bahaddurghatta (Kogunde), Chitradurga Taluk & District - 577519

CC BY-SA 4.0 - Free to reuse with attribution - 2026

Answer Key - Grading Guidelines

Note to Teacher: This answer key provides expected answers and marking guidelines. Accept equivalent answers where appropriate. Partial marks may be awarded for incomplete but correct responses.

Part A: MCQ (10 x 1 = 10 marks)

Q.No	Answer	Explanation
1	c	Merge Sort has $O(n \log n)$ average-case time complexity
2	c	Binary search has $O(\log n)$ time complexity on sorted arrays
3	b	Ambiguity is NOT a characteristic of a good algorithm
4	b	IoT stands for Internet of Things
5	c	Blockchain is the foundation of cryptocurrency
6	b	Flowcharts represent algorithms visually
7	b	On-demand self-service is a characteristic of cloud computing
8	b	Diamond symbol represents decision/condition in flowcharts
9	b	Machine learning is a subset of AI that enables systems to learn from data
10	c	Enabling two-factor authentication is most important cyber safety practice

Part B: Short Answer (8 x 2 = 16 marks)

1. Write algorithm to find largest element and analyse time complexity.

• **Answer:**

Algorithm: FindLargest

1. Set max = first element
2. For each element i from 2 to n :
 - a. If element $i > \text{max}$, set max = element i
3. Return max

Time complexity: $O(n)$ - linear time as we traverse the list once.

- **Marking:** 1 mark for correct algorithm, 1 mark for time complexity analysis.

2. Differentiate between AI, ML, and DL.

- **Answer:** AI is broad field of creating intelligent machines. ML is subset of AI that learns from data without explicit programming. DL is subset of ML using neural networks with multiple layers. Examples: AI - Chess game, ML - Email spam filter, DL - Image recognition.

- **Marking:** 1 mark for explaining each concept, 1 mark for examples.

3. Explain divide and conquer with Merge Sort example.

- **Answer:** Divide and conquer breaks problem into smaller subproblems, solves them recursively, combines solutions. Merge Sort divides array in half recursively until single

elements, then merges sorted halves. Example: [38, 27, 43, 3] -> [38, 27] and [43, 3] -> further division and merging.

- **Marking:** 1 mark for explaining divide and conquer, 1 mark for Merge Sort example.

4. What is Big Data? Explain 5 V's.

- **Answer:** Big Data refers to extremely large datasets that cannot be processed by traditional tools. 5 V's: Volume (massive data size), Velocity (speed of data generation), Variety (different data types), Veracity (data accuracy and quality), Value (useful insights from data).
- **Marking:** 1 mark for Big Data definition, 1 mark for explaining 5 V's.

5. Write Python program for linear and binary search.

- **Answer:**

```
def linear_search(arr, target):
    for i in range(len(arr)):
        if arr[i] == target:
            return i
    return -1
```

```
def binary_search(arr, target):
    low, high = 0, len(arr) - 1
    while low <= high:
        mid = (low + high) // 2
        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            low = mid + 1
        else:
            high = mid - 1
    return -1
```

Comparison: Linear $O(n)$, Binary $O(\log n)$ but requires sorted array

- **Marking:** 1 mark for linear search, 1 mark for binary search.

6. Explain three cyber safety measures.

- **Answer:** 1) Use strong, unique passwords - prevents unauthorized access. 2) Enable two-factor authentication - adds extra security layer. 3) Don't click suspicious links - protects against phishing and malware. Each measure protects different aspects of online safety.
- **Marking:** 1 mark for each measure with explanation.

7. What is cloud computing? Differentiate IaaS, PaaS, SaaS.

- **Answer:** Cloud computing delivers computing services over internet. IaaS: Infrastructure as Service (e.g., AWS EC2 - virtual machines). PaaS: Platform as Service (e.g., Google App Engine - development platform). SaaS: Software as Service (e.g., Google Docs - ready-to-use software).
- **Marking:** 1 mark for cloud computing definition, 1 mark for differentiating IaaS, PaaS, SaaS.

8. Write pseudocode for bubble sort and trace on [5, 3, 8, 1, 2].

- **Answer:**

```

BubbleSort(arr):
  For i = 0 to n-1:
    For j = 0 to n-i-2:
      If arr[j] > arr[j+1]:
        Swap arr[j] and arr[j+1]

```

Trace: Pass 1: [3,5,1,2,8], Pass 2: [3,1,2,5,8], Pass 3: [1,2,3,5,8], Pass 4: [1,2,3,5,8]

- **Marking:** 1 mark for pseudocode, 1 mark for trace.

Part C: Long Answer (4 x 3 = 12 marks)

1. **Write Python program for Insertion Sort with trace and analysis.**
 - **Answer:** Complete program with insertion sort implementation, trace on [12, 11, 13, 5, 6], and analysis: Best case $O(n)$ when sorted, Average/Worst case $O(n^2)$ when reverse sorted.
 - **Marking:** 1 mark for correct program, 1 mark for trace, 1 mark for complexity analysis.
2. **Explain Blockchain, IoT, and AI with examples.**
 - **Answer:** Blockchain: Distributed ledger technology, blocks linked cryptographically, application in supply chain management. IoT: Connected devices exchanging data, example in smart traffic lights. AI: Machines mimicking human intelligence, applications in personalized learning and medical diagnosis.
 - **Marking:** 1 mark for each technology with example.
3. **Design traffic management system using algorithms.**
 - **Answer:** Problem statement, algorithm for signal timing based on vehicle density, flowchart for decision process, IoT sensor integration for real-time data collection and adaptive signal control.
 - **Marking:** 1 mark for problem statement and algorithm, 1 mark for flowchart, 1 mark for IoT integration.
4. **Design library system with algorithms, SQL, and cloud computing.**
 - **Answer:** Search algorithm with error handling, SQL database design for books/members/loans, Python pseudocode for search function, cloud computing for remote access and data backup.
 - **Marking:** 1 mark for algorithm, 1 mark for SQL design, 1 mark for cloud integration.

Part D: Application Based (2 x 1 = 2 marks)

1. **Explain how technology helps social media company.**
 - **Answer:** Big Data analytics processes user behavior patterns to understand preferences. Machine Learning algorithms analyze viewing history to recommend content. AI uses natural language processing to detect and remove harmful content automatically.
 - **Marking:** 1 mark for Big Data and ML explanation, 1 mark for AI explanation.
2. **Explain how technology helps farmer.**
 - **Answer:** IoT sensors monitor soil moisture, temperature, and humidity in real-time. Data analytics processes historical data to predict optimal planting and harvesting times. Cloud computing provides access to weather forecasts and market prices from anywhere.

- **Marking:** 1 mark for IoT and data analytics, 1 mark for cloud computing explanation.

ICT Class 8 - Unit Test 1

Unit Test 1 - Answer Key

- Class 8

Class: Class 8

Assessment Type: Unit Test 1

Total Marks: 10

Duration: 30 minutes

Topics Covered: Functions

Curated and developed by

Hoysala T, Computer Science Teacher

MDRS SC-32 Bahaddurghatta (Kogunde), Chitradurga Taluk & District - 577519

CC BY-SA 4.0 - Free to reuse with attribution - 2026

Answer Key - Grading Guidelines

Note to Teacher: This answer key provides expected answers and marking guidelines. Accept equivalent answers where appropriate. Partial marks may be awarded for incomplete but correct responses.

Section I: Multiple Choice Questions (5 x 1 = 5 marks)

Q.No	Answer	Explanation
1	b	def keyword is used to define functions in Python
2	c	TypeError because add requires 2 arguments but only 1 was provided
3	a	lambda x: x * 2 creates a lambda function
4	b	Default parameter "World" is used when no argument is passed
5	c	return keyword sends a value back from the function

Section II: Short Answer Questions (5 x 1 = 5 marks)

1. Write a Python function `power(base, exp)`.

• **Answer:**

```
def power(base, exp):
    result = 1
    for i in range(exp):
        result *= base
    return result
```

```
print(power(2, 5)) # Output: 32
```

• **Marking:** 1 mark for correct function definition, 1 mark for correct output.

2. What is the output of the global variable modification?

• **Answer:** Output: 15 and 15. The `global` keyword allows the function to modify the global variable `x`. Inside the function, `x` becomes 15, and this change persists outside.

• **Marking:** 1 mark for correct output, 1 mark for explanation.

3. Differentiate between recursion and iteration.

• **Answer:** Recursion is when a function calls itself (e.g., factorial function). Iteration uses loops like `for` or `while` to repeat code (e.g., counting from 1 to 10). Recursion uses stack memory; iteration uses constant memory.

• **Marking:** 1 mark for explaining recursion with example, 1 mark for explaining iteration with example.

4. Write the output of the mystery function.

• **Answer:** "OLLEH" and "NOHTYP". The function reverses the string using slice notation `[::-1]`.

• **Marking:** 1 mark for each correct output.

5. **Explain what happens when a function has no return statement.**

- **Answer:** When a function has no return statement, it returns None by default. Example: `def greet(): print("Hello")` returns None when called. To get a value back, you must use `return`.
- **Marking:** 1 mark for explaining default return value, 1 mark for example.

ICT Class 8 - Unit Test 2

Unit Test 2 - Answer Key

- Class 8

Class: Class 8

Assessment Type: Unit Test 2

Total Marks: 10

Duration: 30 minutes

Topics Covered: Data Structures - Lists, Tuples, Dictionaries

Curated and developed by

Hoysala T, Computer Science Teacher

MDRS SC-32 Bahaddurghatta (Kogunde), Chitradurga Taluk & District - 577519

CC BY-SA 4.0 - Free to reuse with attribution - 2026

Answer Key - Grading Guidelines

Note to Teacher: This answer key provides expected answers and marking guidelines. Accept equivalent answers where appropriate. Partial marks may be awarded for incomplete but correct responses.

Section I: Multiple Choice Questions (5 x 1 = 5 marks)

Q.No	Answer	Explanation
1	a	pop() removes and returns the last element (or specified index)
2	c	TypeError because tuples are immutable and cannot be modified
3	c	Dictionary keys must be immutable (strings, numbers, tuples)
4	c	dict.keys() returns a view object of all keys in the dictionary
5	a	[x**2 for x in range(1,6)] correctly creates list of squares

Section II: Short Answer Questions (5 x 1 = 5 marks)

1. Write the output and explain what happened.

- **Answer:** Output: [10, 40, 50]. The `del nums[1:3]` statement deleted elements at indices 1 and 2 (values 20 and 30) from the list.
- **Marking:** 1 mark for correct output, 1 mark for explanation.

2. Write a Python program to merge two dictionaries.

- **Answer:**

```
d1 = {"a": 1, "b": 2}
d2 = {"c": 3, "d": 4}
```

```
# Method 1: Using update()
merged = d1.copy()
merged.update(d2)
```

```
# Method 2: Using unpacking (Python 3.5+)
# merged = {**d1, **d2}
```

```
print(merged)
```

- **Marking:** 1 mark for correct dictionary creation, 1 mark for correct merge operation.

3. What will be the output and why?

- **Answer:** Output: 2 4 6. The loop iterates through each row in the nested list, and `row[1]` accesses the second element of each inner list.
- **Marking:** 1 mark for correct output, 1 mark for explanation.

4. Differentiate between `list.remove(x)` and `list.pop(index)`.

- **Answer:** `remove(x)` searches for the first occurrence of value `x` and removes it; returns nothing. `pop(index)` removes and returns the element at specified index; if no index given, removes last element.
- **Marking:** 1 mark for explaining `remove`, 1 mark for explaining `pop`.

5. **Write a program to count character frequency.**

- **Answer:**

```
def char_frequency(text):  
    freq = {}  
    for char in text:  
        freq[char] = freq.get(char, 0) + 1  
    # Sort by frequency  
    sorted_freq = sorted(freq.items(), key=lambda x: x[1], reverse=True)  
    return sorted_freq  
  
text = "hello world"  
for char, count in char_frequency(text):  
    print(f"'{char}': {count}")
```

- **Marking:** 1 mark for dictionary creation, 1 mark for frequency counting and sorting.

ICT Class 8 - Unit Test 3

Unit Test 3 - Answer Key

- Class 8

Class: Class 8

Assessment Type: Unit Test 3

Total Marks: 10

Duration: 30 minutes

Topics Covered: SQL - Database Fundamentals

Curated and developed by

Hoysala T, Computer Science Teacher

MDRS SC-32 Bahaddurghatta (Kogunde), Chitradurga Taluk & District - 577519

CC BY-SA 4.0 - Free to reuse with attribution - 2026

Answer Key - Grading Guidelines

Note to Teacher: This answer key provides expected answers and marking guidelines. Accept equivalent answers where appropriate. Partial marks may be awarded for incomplete but correct responses.

Section I: Multiple Choice Questions (5 x 1 = 5 marks)

Q.No	Answer	Explanation
1	b	COUNT() returns the total number of rows
2	c	HAVING clause filters groups after GROUP BY
3	b	DISTINCT returns unique department values only
4	a	ALTER TABLE is used to modify existing table structure
5	a	Option a has correct syntax: SELECT * FROM Students WHERE marks > 80 AND grade = 'A';

Section II: Short Answer Questions (5 x 1 = 5 marks)

1. Write a SQL query to create a table Products.

• **Answer:**

```
CREATE TABLE Products (
    id INT PRIMARY KEY,
    name VARCHAR(50),
    price DECIMAL(10, 2),
    quantity INT
);
```

• **Marking:** 1 mark for correct CREATE TABLE syntax, 1 mark for all columns with data types.

2. Write SQL queries to insert 3 records and update price.

• **Answer:**

```
INSERT INTO Products VALUES (1, 'Laptop', 45000.00, 10);
INSERT INTO Products VALUES (2, 'Mouse', 500.00, 50);
INSERT INTO Products VALUES (3, 'Keyboard', 1200.00, 30);
```

```
UPDATE Products SET price = 450.00 WHERE id = 2;
```

• **Marking:** 1 mark for correct INSERT statements, 1 mark for correct UPDATE.

3. What is the difference between WHERE and HAVING?

• **Answer:** WHERE filters rows before grouping (cannot use aggregate functions). HAVING filters groups after GROUP BY (can use aggregate functions). Example: WHERE salary > 50000 vs HAVING AVG(salary) > 40000.

• **Marking:** 1 mark for explaining WHERE, 1 mark for explaining HAVING with examples.

4. Write a query to find average price grouped by quantity.

• **Answer:**

```
SELECT quantity, AVG(price) as avg_price  
FROM Products  
GROUP BY quantity;
```

- **Marking:** 1 mark for correct GROUP BY, 1 mark for AVG function.

5. **Write a query to display top 3 products by price.**

- **Answer:**

```
SELECT * FROM Products  
ORDER BY price DESC  
LIMIT 3;
```

- **Marking:** 1 mark for ORDER BY DESC, 1 mark for LIMIT clause.

ICT Class 8 - Unit Test 4

Unit Test 4 - Answer Key

- Class 8

Class: Class 8

Assessment Type: Unit Test 4

Total Marks: 10

Duration: 30 minutes

Topics Covered: HTML and CSS

Curated and developed by

Hoysala T, Computer Science Teacher

MDRS SC-32 Bahaddurghatta (Kogunde), Chitradurga Taluk & District - 577519

CC BY-SA 4.0 - Free to reuse with attribution - 2026

Answer Key - Grading Guidelines

Note to Teacher: This answer key provides expected answers and marking guidelines. Accept equivalent answers where appropriate. Partial marks may be awarded for incomplete but correct responses.

Section I: Multiple Choice Questions (5 x 1 = 5 marks)

Q.No	Answer	Explanation
1	c	<h1> defines the largest heading in HTML
2	b	font-size property controls the size of text in CSS
3	b	CSS stands for Cascading Style Sheets
4	c	style attribute is used to specify inline styles in HTML
5	b	background-color property changes the background colour

Section II: Short Answer Questions (5 x 1 = 5 marks)

1. Write HTML code for an unordered list with class menu-list.

• **Answer:**

```
<ul class="menu-list">
  <li>Home</li>
  <li>About</li>
  <li>Services</li>
  <li>Contact</li>
</ul>
```

• **Marking:** 1 mark for correct structure, 1 mark for class attribute.

2. Explain the difference between id and class selectors.

• **Answer:** id selector (#header) targets a single unique element (used once per page). class selector (.menu) targets multiple elements with the same class (can be reused). Example:

```
<div id="main"> VS <p class="text">.
```

• **Marking:** 1 mark for explaining id, 1 mark for explaining class with examples.

3. Write CSS to style a button with specific properties.

• **Answer:**

```
button {
  padding: 10px 20px;
  background-color: blue;
  color: white;
  border-radius: 5px;
  border: none;
  cursor: pointer;
}
```

```
button:hover {
```



```
background-color: darkblue;
}
```

- **Marking:** 1 mark for basic button styling, 1 mark for hover effect.

4. **What is the purpose of the `<!DOCTYPE html>` declaration?**

- **Answer:** The `<!DOCTYPE html>` declaration tells the browser that the document is an HTML5 document. It must be the first line in an HTML document to ensure the browser renders the page in standards mode.
- **Marking:** 1 mark for explaining purpose, 1 mark for importance.

5. **Write HTML and CSS code for a card layout.**

- **Answer:**

```
<div class="card">
  
  <div class="card-content">
    <h3 class="card-title">Card Title</h3>
    <p class="card-text">This is a description of the card content.</p>
    <a href="#" class="card-link">Read More</a>
  </div>
</div>

<style>
.card {
  border: 1px solid #ddd;
  border-radius: 8px;
  overflow: hidden;
  max-width: 300px;
}
.card-image {
  width: 100%;
  height: 200px;
  object-fit: cover;
}
.card-content {
  padding: 15px;
}
.card-title {
  margin: 0 0 10px 0;
}
.card-link {
  display: inline-block;
  margin-top: 10px;
  color: blue;
  text-decoration: none;
}
</style>
```

- **Marking:** 1 mark for HTML structure, 1 mark for CSS styling.